PROJECT REPORT

Inventory Management System

Student Name: Arnav Singh Kushwah

Registration Number: 24BAI10416

Program: B.Tech — Computer Science and Engineering(AIML)

Academic Year: 2026

Project Title: Inventory Management System — A Desktop Application for Small Business Operations

Technology Stack: Java SE, Java Swing, SQLite, JDBC, SHA-256 Cryptographic Hashing

Table of Contents

1. Executive Summary
2. Introduction and Background
3. Problem Statement — In Depth
4. Why This Problem Matters
5. Objectives of the Project
6. Scope of the Project
7. System Requirements
8. Approach to Solving the Problem
9. System Architecture and Design
10. Database Design and Schema
11. Module-by-Module Implementation
12. Key Design Decisions
13. Challenges Faced and How They Were Resolved
14. Testing and Validation
15. What I Learned
16. Future Improvements and Roadmap
17. Conclusion
18. References

1. Executive Summary

This report documents the complete design, development, and evaluation of an Inventory Management System built as a desktop application using Java. The system was conceived to address a very real and widespread problem faced by small and medium-sized businesses across India and globally — the absence of an affordable, reliable, offline-capable tool to manage product stock, process customer sales, track financial transactions, and generate business reports in one unified platform.

The application was developed entirely in Java using the Swing framework for the graphical user interface and SQLite as the backend database, connected through the JDBC API. The system supports multiple user accounts with secure SHA-256 hashed credentials, a full product catalog with low-stock alerting, a complete sales and billing engine with automatic tax calculation and receipt generation, a double-entry-style accounts ledger for income and expense tracking, and a reports module for business analytics.

The project was built from scratch over the course of the academic semester, starting from requirements gathering and database schema design, through to full GUI implementation and integration testing. Every module was designed with separation of concerns in mind — business logic lives in dedicated manager classes, data access is centralized in a single connection class, and the GUI layer only handles presentation and user interaction.

This report covers every aspect of the project in detail: the problem that motivated it, the technical decisions made

during development, the architecture of the system, the implementation of each module, the challenges encountered and how they were resolved, and the lessons learned throughout the process. It is intended to serve as a complete technical and reflective account of the project for academic evaluation.

2. Introduction and Background

2.1 The Context of Small Business Operations in India

India has one of the largest small business ecosystems in the world. According to government data, there are over 63 million micro, small, and medium enterprises (MSMEs) operating across the country, employing hundreds of millions of people and contributing significantly to the national GDP. These businesses span every sector — retail shops, pharmacies, grocery stores, hardware stores, clothing boutiques, electronics dealers, warehouses, and more.

Despite their enormous economic importance, the vast majority of these businesses operate with extremely limited technological infrastructure. Most small shop owners still rely on handwritten ledgers, paper bills, and mental arithmetic to manage their day-to-day operations. Those who have adopted some form of digital record-keeping typically use basic spreadsheet software like Microsoft Excel or Google Sheets, which, while better than paper, are not purpose-built for inventory management and lack features like automatic stock deduction on sale, low-stock alerts, or integrated billing.

The reasons for this technology gap are well understood. Enterprise-grade inventory and ERP software is expensive — both in terms of licensing fees and the technical expertise required to set it up and maintain it. Cloud-based SaaS solutions require a reliable internet connection, which is not always available in smaller towns and semi-urban areas. And most off-the-shelf solutions are designed for larger

businesses, making them unnecessarily complex and overwhelming for a shop owner managing a few hundred products.

This is the context in which this project was conceived. The goal was to build something that fills this gap — a free, open-source, offline-capable desktop application that any small business owner can install on a basic computer and start using immediately, without needing technical expertise, an internet connection, or a subscription fee.

2.2 What is an Inventory Management System

An Inventory Management System, at its core, is a software tool that helps a business keep track of its goods — what it has, how much of it there is, where it came from, and where it went. In a retail context, this means tracking products from the moment they are added to the catalog, through every sale that reduces their stock, to the point where they need to be reordered.

A well-designed inventory management system does more than just count stock. It integrates with the billing process so that every sale automatically updates stock levels. It alerts the business owner when a product is running low. It tracks the financial side of the business — recording income from sales and expenses from purchases and overheads. It generates reports that help the owner understand which products are selling well, which are sitting idle, and whether the business is profitable.

The system built in this project covers all of these functions. It is not a simple stock counter — it is a complete business

operations tool that handles user authentication, product management, sales processing, financial accounting, and reporting in a single integrated application.

2.3 Why Java and Desktop

The choice of Java as the implementation language and desktop as the deployment target was deliberate and well-reasoned. Java is platform-independent — a Java application compiled once can run on Windows, macOS, and Linux without modification, as long as the Java Runtime Environment is installed. This is critical for a tool intended for small businesses, which may be running any operating system on their computers.

Java Swing, while considered a mature (some would say older) GUI framework, is extremely capable for building desktop applications. It provides a rich set of UI components, supports complex layouts, and does not require any external dependencies beyond the JDK itself. For a project of this scope — a multi-screen, multi-module desktop application — Swing is entirely appropriate and produces a professional-looking result.

SQLite was chosen as the database because it requires absolutely no server setup. The entire database is a single file on disk. This makes the application truly self-contained and portable. A business owner can back up their entire database by simply copying one file. There is no database server to install, configure, or maintain.

3. Problem Statement — In Depth

3.1 The Core Problem

The fundamental problem this project addresses is the lack of an integrated, affordable, and accessible inventory management solution for small businesses. This problem manifests in several specific, concrete ways that have real financial consequences for business owners.

Stock Discrepancies and Shrinkage

When stock is tracked manually — either on paper or in a spreadsheet that is updated inconsistently — discrepancies accumulate over time. A product is sold but the ledger is not updated. A delivery arrives but the count is entered incorrectly. An item is returned but the stock is not restored. Over weeks and months, these small errors compound into significant discrepancies between what the records say and what is actually on the shelf. This leads to situations where a business thinks it has stock it does not have, resulting in failed sales and disappointed customers, or where it has stock it does not know about, resulting in unnecessary reorders and wasted capital.

Missed Reorder Points

Without a system that tracks stock levels and compares them against minimum thresholds, business owners often do not realize a product is running low until it has completely run out. By the time a reorder is placed and the new stock arrives, the business has lost sales and potentially lost customers to competitors. This is particularly damaging for

fast-moving consumer goods where customers expect consistent availability.

Inefficient Billing Process

Manual billing — writing out receipts by hand or using a basic calculator — is slow, error-prone, and unprofessional. It is easy to make arithmetic mistakes, forget to apply tax, or give the wrong change. It also means that sales data is not captured in any structured way, making it impossible to analyze sales patterns or generate reports later.

No Financial Visibility

Most small business owners have a very poor understanding of their actual financial position. They know roughly how much cash they have in the till, but they do not have a clear picture of total income, total expenses, net profit, or cash flow trends. Without this visibility, it is impossible to make informed business decisions — whether to expand, whether to cut costs, whether a particular product line is profitable.

No Audit Trail

Paper-based systems provide no reliable audit trail. If a dispute arises about a past transaction, there is no way to look it up. If an employee makes an error or, worse, engages in theft, there is no record to investigate. A digital system with timestamped records for every transaction provides accountability and traceability that paper simply cannot.

3.2 Who This Problem Affects

This problem affects an enormous number of businesses. In the Indian context alone, the target users include:

- Kirana stores and general merchandise shops
- Medical and pharmaceutical retailers
- Electronics and mobile phone shops
- Clothing and textile retailers
- Hardware and building materials suppliers
- Stationery and book shops
- Grocery stores and supermarkets
- Small warehouses and distribution centers
- Restaurants and food service businesses that track ingredient inventory

Each of these business types has slightly different needs, but they all share the core requirement: a way to track what they have, record what they sell, and understand their financial position.

3.3 Why Existing Solutions Fall Short

It is worth examining why existing solutions do not adequately address this problem, because understanding the gap helps explain the design decisions made in this project.

Enterprise ERP Systems such as SAP, Oracle, and Microsoft Dynamics are powerful but are designed for large

organizations. They require dedicated IT staff to install and maintain, cost thousands of dollars in licensing fees, and have interfaces so complex that they require weeks of training. They are completely inaccessible to a small shop owner.

Cloud-based SaaS Inventory Tools such as Zoho Inventory, QuickBooks, and similar products are more accessible but still have significant drawbacks for the target user. They require a monthly subscription fee, which is a recurring cost that many small businesses cannot justify. They require a reliable internet connection at all times, which is not guaranteed in many parts of India. And they store business data on remote servers, which raises privacy and data sovereignty concerns for some business owners.

Spreadsheet-based Solutions using Excel or Google Sheets are free and familiar but are fundamentally not designed for this purpose. They have no built-in concept of a product catalog, no automatic stock deduction on sale, no bill generation, no low-stock alerts, and no financial reporting. Every feature has to be manually built using formulas, which requires technical skill and breaks easily.

Basic POS Software available in the market is often designed only for billing and does not include inventory tracking, financial accounting, or reporting. It solves one part of the problem but not the whole.

This project fills the gap by providing a purpose-built, integrated, offline-capable, free desktop application that handles all of these needs in one place.

4. Why This Problem Matters

4.1 Economic Impact of Poor Inventory Management

The financial consequences of poor inventory management are well-documented and significant. Research in retail operations consistently shows that inventory-related problems — stockouts, overstocking, shrinkage, and billing errors — account for a substantial portion of revenue loss in small retail businesses.

A stockout — running out of a product a customer wants to buy — results in an immediate lost sale. But the damage goes beyond that single transaction. Studies show that a significant percentage of customers who encounter a stockout will not wait for the product to be restocked; they will go to a competitor. If this happens repeatedly, the business loses not just individual sales but entire customer relationships.

Overstocking is the opposite problem but equally damaging. Capital tied up in excess inventory is capital that cannot be used for other purposes — paying staff, investing in marketing, or expanding the product range. Overstocked perishable goods may expire and have to be written off entirely. Even non-perishable goods that sit on shelves for too long represent a hidden cost in storage space and tied-up capital.

Billing errors — whether from arithmetic mistakes or from applying the wrong price — erode profit margins and damage customer trust. A customer who is overcharged and

notices it will lose confidence in the business. A customer who is undercharged means the business is giving away margin it cannot afford to lose.

4.2 The Broader Significance

Beyond the direct financial impact on individual businesses, there is a broader significance to solving this problem well. When small businesses operate more efficiently, they are more likely to survive and grow. Business survival and growth means more employment, more economic activity, and more contribution to the local and national economy.

There is also a digital inclusion dimension to this problem. One of the barriers to small businesses adopting technology is the perception that technology is complicated, expensive, and not designed for them. A well-designed, accessible, free tool that genuinely solves their problems can change that perception and open the door to further digital adoption — better record-keeping, online presence, digital payments, and eventually more sophisticated analytics.

4.3 Academic and Technical Significance

From an academic perspective, this project is significant because it demonstrates the application of multiple computer science concepts in an integrated, real-world context. It is not a toy application or a tutorial exercise — it is a system that could genuinely be used by a real business. Building it required applying knowledge from multiple domains: object-oriented programming, database design, GUI development, security (cryptographic hashing), software architecture (layered design, separation of

concerns), and software engineering practices (transaction management, error handling, input validation).

The project also demonstrates the value of understanding the problem domain deeply before writing code. The design of the database schema, the choice of which features to include, and the decisions about how modules interact with each other were all informed by a genuine understanding of how small businesses operate and what they need.

5. Objectives of the Project

The following objectives were defined at the start of the project and guided all subsequent design and implementation decisions.

Primary Objectives

The first primary objective was to build a fully functional, multi-module desktop application that integrates product management, sales processing, financial accounting, and reporting in a single cohesive system. The application needed to be genuinely usable — not just a prototype or a demonstration, but something with real functionality that works end-to-end.

The second primary objective was to implement a secure user authentication system that protects business data from unauthorized access. This meant not just a simple username/password check, but proper cryptographic hashing of credentials, a password recovery mechanism, and session management within the application.

The third primary objective was to design and implement a relational database schema that accurately models the business domain — products, sales, sale items, financial transactions, and users — with appropriate relationships, constraints, and data types.

Secondary Objectives

The first secondary objective was to build a clean, intuitive graphical user interface that a non-technical user could navigate without training. The interface needed to be visually organized, with clear navigation, consistent styling, and informative feedback messages.

The second secondary objective was to implement the system with a clean layered architecture that separates the GUI, business logic, and data access layers. This was important both for code quality and for demonstrating good software engineering practice.

The third secondary objective was to ensure data integrity through the use of database transactions, particularly in the sales processing module where multiple tables need to be updated atomically.

The fourth secondary objective was to implement a low-stock alerting system that proactively notifies the user when products fall below their defined minimum stock threshold.

Learning Objectives

Beyond the functional objectives, the project was also intended to achieve specific learning outcomes: gaining hands-on experience with JDBC and SQL in a real application context, understanding the practical implications of cryptographic security in software, learning to design and build a multi-screen Swing application, and developing the ability to think through a complex problem domain and translate it into a working software system.

6. Scope of the Project

6.1 What is Included

The scope of this project includes the following:

The user authentication module covers registration of new user accounts, login with credential verification, a forgot-password flow using security questions, temporary password generation, and password reset. All passwords and security answers are stored as SHA-256 hashes.

The product management module covers adding new products to the catalog with full details (name, description, category, selling price, cost price, stock quantity, and minimum stock level), editing existing product records, deleting products, viewing all products in a tabular format, and identifying products that are at or below their minimum stock threshold.

The sales and billing module covers creating new sales by selecting products and quantities, entering customer details, choosing a payment method, calculating subtotals with automatic 10% tax and optional discount, generating a formatted bill receipt, recording the sale in the database, automatically decrementing product stock quantities, and automatically recording the sale amount as income in the accounts ledger. It also covers viewing sales history with date-range filtering.

The account management module covers recording income and expense transactions with description and category, filtering transactions by type and time period, and viewing a financial summary showing total income, total expenses, net profit, and current balance.

The reports module covers a sales report with date filtering, an inventory report showing stock levels, and a financial report with income and expense breakdown.

The dashboard provides a central navigation hub with quick-access buttons to all modules and a stats bar showing key business metrics.

6.2 What is Not Included

The following items are explicitly outside the scope of this version of the project:

The system does not include supplier management — there is no module for recording purchase orders, tracking suppliers, or managing incoming stock deliveries. Stock can be updated manually through the product edit function, but there is no formal procurement workflow.

The system does not include customer relationship management. Customer names are recorded on sales, but there is no customer database, no purchase history per customer, and no loyalty or discount management.

The system does not include multi-location or multi-branch support. The system is designed for a single business location.

The system does not include barcode scanning. Products must be selected manually from the catalog during a sale.

The system does not include PDF export of bills or reports. Bill receipts are displayed on screen but cannot be printed or saved as PDF in this version.

The system does not include data visualization charts. The reports module has placeholder panels where charts would appear in a future version.

The system does not include role-based access control. All authenticated users have full access to all features.

7. System Requirements

7.1 Functional Requirements

FR-01: The system shall allow new users to register by providing a username, password, email address, a custom security question, and a security answer. The system shall reject registration if the username already exists in the database.

FR-02: The system shall allow registered users to log in using their username and password. The system shall hash the entered password using SHA-256 and compare it against the stored hash. Login shall be denied if the credentials do not match.

FR-03: The system shall provide a forgot-password workflow. A user shall be able to enter their username, retrieve their security question, provide the correct answer, and receive a system-generated temporary password that replaces their current password.

FR-04: The system shall allow authenticated users to add new products to the catalog. Each product record shall store a name, description, category, selling price, cost price, current stock quantity, and minimum stock level.

FR-05: The system shall allow authenticated users to edit any field of an existing product record and save the changes to the database with an updated timestamp.

FR-06: The system shall allow authenticated users to delete a product from the catalog. The system shall confirm the deletion before executing it.

FR-07: The system shall display all products in a tabular view with columns for ID, name, category, price, cost, stock quantity, and minimum stock level.

FR-08: The system shall identify and display all products whose current stock quantity is less than or equal to their minimum stock level, presenting them as a low-stock alert.

FR-09: The system shall allow authenticated users to create a new sale by selecting one or more products, specifying quantities, entering a customer name, and selecting a payment method.

FR-10: The system shall automatically calculate the subtotal of a sale, apply a 10% tax, apply any entered discount, and compute the final payable amount.

FR-11: Upon completing a sale, the system shall generate a unique bill number based on the current timestamp, insert a sale record into the sales table, insert individual line items into the sale_items table, decrement the stock quantity of each sold product in the products table, and insert an income entry into the accounts table. All of these operations shall be performed within a single database transaction.

FR-12: The system shall display a formatted bill receipt showing the bill number, date, customer name, itemized list of products with quantities and prices, subtotal, tax, discount, total, and payment method.

FR-13: The system shall allow users to view sales history with the ability to filter by date range.

FR-14: The system shall allow users to record financial transactions of type income or expense, with an amount, description, and category.

FR-15: The system shall allow users to filter transactions by type and by time period (today, this week, this month, or all time).

FR-16: The system shall display a financial summary showing total income, total expenses, net profit, and current balance, calculated from the accounts table.

FR-17: The system shall provide a reports section with tabs for sales reports, inventory reports, and financial reports.

FR-18: The system shall display a dashboard with navigation buttons to all modules and a quick stats bar showing total products, low stock count, today's sales, and monthly revenue.

7.2 Non-Functional Requirements

NFR-01 Security: All passwords and security answers shall be stored as SHA-256 hashes. No plaintext credentials shall ever be written to the database.

NFR-02 Data Integrity: All multi-table write operations, particularly sale creation, shall be wrapped in database transactions with rollback on failure to prevent partial or inconsistent data.

NFR-03 Performance: The application shall load any screen within two seconds on a standard modern computer. Database queries shall be optimized to avoid full table scans where possible.

NFR-04 Usability: The interface shall be navigable by a non-technical user without any training manual. All actions shall provide clear feedback messages indicating success or failure.

NFR-05 Portability: The application shall run on any operating system that supports Java SE 8 or higher, including Windows, macOS, and Linux.

NFR-06 Reliability: The application shall handle database errors gracefully, displaying user-friendly error messages rather than crashing or showing raw stack traces.

NFR-07 Maintainability: The codebase shall follow a layered architecture with clear separation between the GUI layer, business logic layer, and data access layer. Each module shall be implemented in its own class.

7.3 Hardware and Software Requirements

Minimum hardware requirements for running the application are a processor of 1 GHz or faster, 512 MB of RAM, and 100

MB of available disk space for the application and database. A display resolution of at least 1024x768 is recommended for the GUI to render correctly.

Software requirements are Java SE Runtime Environment version 8 or higher, and the SQLite JDBC driver JAR file placed in the application's lib directory. No other software installation is required.

8. Approach to Solving the Problem

8.1 Planning and Requirements Analysis

The development process began with a thorough analysis of the problem domain. Before writing a single line of code, time was spent understanding what a small business owner actually needs from an inventory management tool. This involved thinking through the daily workflows of a typical retail shop: receiving stock, adding new products, serving customers, generating bills, recording expenses, and reviewing the day's performance.

From this analysis, a list of core features was derived. The features were then prioritized into must-have (core functionality without which the application is not useful), should-have (important features that significantly improve usability), and nice-to-have (features that add value but are not essential for the first version). This prioritization ensured that development effort was focused on the most important functionality first.

The must-have features identified were: user authentication, product CRUD operations, sale creation with automatic stock update, bill generation, and basic financial transaction recording. The should-have features were: low-stock alerts, sales history with filtering, financial summary dashboard, and a reports section. The nice-to-have features — PDF export, charts, barcode scanning, supplier management — were noted for future development but excluded from the initial scope.

8.2 Database-First Design

The decision was made to design the database schema before designing the GUI or writing any application logic. This database-first approach is sound software engineering practice because the database schema is the foundation of the entire system. If the schema is wrong, everything built on top of it will be wrong too.

The schema design process involved identifying the core entities in the business domain — users, products, sales, sale line items, and financial transactions — and defining the attributes of each entity and the relationships between them. The relationship between sales and sale_items is a classic one-to-many relationship: one sale has many line items. The relationship between sale_items and products is a many-to-one reference: many sale items can reference the same product.

Foreign key constraints were defined between sale_items and both sales and products to enforce referential integrity at the database level. This means the database itself will reject any attempt to insert a sale item that references a non-existent sale or product.

8.3 Layered Architecture Design

With the database schema defined, the next step was to design the application architecture. A three-layer architecture was chosen: a presentation layer (the GUI), a business logic layer (the manager classes), and a data access layer (the DatabaseConnection class and the SQL within the manager classes).

This separation of concerns means that the GUI classes do not contain any SQL. They call methods on the manager classes, which handle all database interaction. This makes the code much easier to understand, debug, and modify. If the database schema changes, only the manager classes need to be updated — the GUI code remains untouched.

8.4 Iterative Development

Development proceeded iteratively, building and testing one module at a time. The first module built was DatabaseConnection, because everything else depends on it. Once the database initialization was working correctly, UserAuth was built and tested. Then ProductManager, then SalesManager, then AccountManager. The GUI was built last, wiring together all the backend modules.

This bottom-up approach meant that by the time the GUI was being built, all the underlying logic had already been tested and was known to work. GUI development could focus entirely on layout and user interaction without worrying about whether the business logic was correct.

9. System Architecture and Design

9.1 Overall Architecture

The system follows a classic three-tier desktop application architecture. The three tiers are the presentation tier, the logic tier, and the data tier.

The presentation tier consists of the Java Swing GUI classes — Main.java and InventoryManagementGUI.java. These classes are responsible for rendering the user interface, capturing user input, and displaying results. They contain no business logic and no SQL. Their only job is to present information to the user and translate user actions into calls to the logic tier.

The logic tier consists of the four manager classes: UserAuth.java, ProductManager.java, SalesManager.java, and AccountManager.java. Each manager class is responsible for one domain of the application. These classes contain all the business rules — how a sale is calculated, how a password is hashed, what constitutes a low-stock product — and all the SQL queries that implement those rules.

The data tier consists of DatabaseConnection.java and the SQLite database file inventory.db. DatabaseConnection provides a single method for obtaining a database connection and a method for initializing the schema. All actual data storage and retrieval happens through this tier.

9.2 Class Responsibilities

DatabaseConnection.java is responsible for two things only: providing a JDBC connection to the SQLite database, and initializing the database schema on first run. The connection URL is defined as a constant pointing to a local file called inventory.db. The initializeDatabase method executes CREATE TABLE IF NOT EXISTS statements for all five tables, ensuring the schema is set up correctly without failing if the tables already exist.

UserAuth.java handles all authentication-related operations. It provides static methods for registering a user, logging in, retrieving a security question, verifying a security answer, resetting a password, and generating a temporary password. It also contains the private hashPassword method that implements SHA-256 hashing using Java's MessageDigest class. All methods that store or compare credentials call hashPassword before interacting with the database.

ProductManager.java handles all product-related operations. It provides static methods for adding a product, updating a product, deleting a product, retrieving all products, retrieving a single product by ID, updating a product's quantity (used during sale processing), and retrieving all low-stock products. It also defines the Product inner class that serves as a data transfer object for product records.

SalesManager.java handles all sales-related operations. It provides static methods for generating a bill number, creating a complete sale (the most complex operation in the system), retrieving sales within a date range, retrieving the line items for a specific sale, and generating a formatted text

receipt. It defines the Sale and SaleItem inner classes as data transfer objects.

AccountManager.java handles all financial transaction operations. It provides static methods for adding a transaction, retrieving transactions with optional type and period filters, getting the current balance, and getting total income or expense for a given period. It defines the AccountTransaction inner class as a data transfer object.

Main.java (and its near-identical counterpart InventoryManagementGUI.java) is the GUI class. It creates the main application window, sets up the CardLayout panel that holds all screens, and creates each screen as a separate panel. It contains helper methods for creating styled buttons, form fields, and header panels, and navigation methods for switching between screens.

9.3 Navigation Flow

The application uses a CardLayout to manage screen navigation. All screens are created at startup and added to the main panel with string identifiers. Switching screens is done by calling cardLayout.show(mainPanel, screenName), which instantly swaps the visible panel without creating new windows or destroying existing ones.

The navigation flow is as follows. The application starts on the LOGIN screen. From LOGIN, the user can go to REGISTER or FORGOT_PASSWORD. Both of those screens have a back button that returns to LOGIN. After a successful login, the user is taken to DASHBOARD. From DASHBOARD, the user can navigate to PRODUCTS, SALES, ACCOUNTS, or

REPORTS. Each of those screens has a back button that returns to DASHBOARD. The user can also log out from DASHBOARD, which clears the session and returns to LOGIN.

9.4 Color Design System

The GUI uses a consistent color palette defined as constants in the GUI class. The primary color is a deep blue (RGB 41, 128, 185) used for headers and primary actions. The secondary color is a lighter blue (RGB 52, 152, 219) used for secondary actions and accents. The accent color is a red (RGB 231, 76, 60) used for destructive actions like delete and logout. The success color is a green (RGB 46, 204, 113) used for positive actions like save and confirm. The warning color is a yellow (RGB 241, 196, 15) used for alerts and caution actions. The light background color is a very light grey (RGB 236, 240, 241) used as the background for all panels.

This consistent color system gives the application a professional, cohesive visual identity and helps users quickly understand the nature of each action — green means go, red means danger, yellow means caution.

10. Database Design and Schema

10.1 Design Principles

The database schema was designed following standard relational database design principles. Each table represents a single entity or relationship. Each row in a table represents a single instance of that entity. Columns are atomic — each column stores a single value of a defined type. Primary keys uniquely identify each row. Foreign keys enforce relationships between tables. Default values are used where appropriate to reduce the burden on the application code.

The schema uses SQLite's native data types: INTEGER for whole numbers, REAL for floating-point numbers, TEXT for strings, and DATETIME for timestamps. SQLite's AUTOINCREMENT keyword is used for primary keys to ensure each new record gets a unique identifier automatically.

10.2 The Users Table

The users table stores all registered user accounts. The id column is an auto-incremented integer primary key. The name column stores the username and has a UNIQUE constraint, ensuring no two users can have the same username. The password column stores the SHA-256 hash of the user's password — never the plaintext password itself. The email column stores the user's email address. The security_question column stores the text of the user's chosen security question. The security_answer column stores the SHA-256 hash of the security answer. The

created_at column stores the timestamp of when the account was created, defaulting to the current time.

The UNIQUE constraint on the name column is enforced at the database level, meaning even if the application code fails to check for duplicate usernames, the database will reject the insert and throw a SQLException that the application catches and handles gracefully.

10.3 The Products Table

The products table stores the product catalog. The id column is the auto-incremented primary key. The name column stores the product name. The description column stores an optional longer description. The category column stores the product category, which can be used for filtering and reporting. The price column stores the selling price as a REAL (floating-point) number. The cost column stores the purchase or cost price, also as REAL. The quantity column stores the current stock level as an INTEGER. The min_stock_level column stores the minimum threshold below which a low-stock alert should be triggered, also INTEGER, defaulting to 0. The created_at and updated_at columns store creation and last-modification timestamps.

The separation of price and cost is important because it allows the system to calculate profit margin per product — the difference between what the business charges customers and what it paid to acquire the product. This data is available for reporting even though the current version does not yet surface it in a dedicated margin report.

10.4 The Sales Table

The sales table stores one record per completed sale transaction. The id is the auto-incremented primary key. The bill_number column stores a unique human-readable identifier for the bill, generated by the application as "BILL-" followed by a timestamp in yyyyMMddHHmmss format. The UNIQUE constraint on bill_number ensures no two bills can have the same number. The customer_name column stores the customer's name as provided at the time of sale. The total_amount column stores the subtotal before tax and discount. The tax_amount column stores the calculated tax. The discount column stores any discount applied, defaulting to 0. The final_amount column stores the actual amount charged to the customer after tax and discount. The payment_method column stores the payment type. The sale_date column stores the timestamp of the sale.

10.5 The Sale Items Table

The sale_items table stores the individual line items that make up each sale. This is a child table to the sales table — every row in sale_items belongs to exactly one row in sales. The id is the auto-incremented primary key. The sale_id column is a foreign key referencing sales.id. The product_id column is a foreign key referencing products.id. The product_name column stores the product name at the time of sale — this is important because if a product's name is later changed, the historical bill should still show the original name. The quantity column stores how many units were sold. The unit_price column stores the price per unit at the time of sale, again captured at sale time to preserve historical accuracy. The total_price column stores quantity multiplied by unit_price.

The foreign key constraints on sale_id and product_id enforce referential integrity. SQLite supports foreign keys but requires them to be enabled explicitly with PRAGMA foreign_keys = ON, which should be set when establishing each connection.

10.6 The Accounts Table

The accounts table serves as the financial ledger for the business. Every financial event — whether a sale income entry, a manually recorded expense, or any other transaction — is stored here. The id is the auto-incremented primary key. The transaction_type column stores either "income" or "expense". The amount column stores the transaction amount as REAL. The description column stores a human-readable description of the transaction. The category column stores a category label for grouping and filtering. The transaction_date column stores the timestamp, defaulting to the current time.

The financial summary calculations — total income, total expenses, net profit, and balance — are all derived from this table using SQL aggregate functions (SUM with CASE expressions), which is efficient and ensures the summary is always consistent with the underlying transaction data.

10.7 Entity Relationship Summary

The relationships between tables are as follows. The users table is standalone — it has no foreign key relationships with other tables in this version. The products table is standalone in terms of foreign keys but is referenced by sale_items. The sales table is the parent of sale_items — one sale has many

sale items. The sale_items table references both sales and products. The accounts table is standalone — sale income entries are inserted programmatically by the application when a sale is completed, creating a logical link between sales and accounts even though there is no formal foreign key.

11. Module-by-Module Implementation

11.1 DatabaseConnection Module

The DatabaseConnection class is the foundation of the entire data layer. It contains two public static methods and one private constant.

The URL constant is defined as "jdbc:sqlite:inventory.db". This tells the JDBC driver to connect to a SQLite database stored in a file called inventory.db in the current working directory. When the application is run, this file is created automatically if it does not exist.

The connect() method calls DriverManager.getConnection(URL) and returns the resulting Connection object. It catches SQLException and prints an error message if the connection fails, returning null in that case. All calling code must handle the possibility of a null connection. The method is called with try-with-resources throughout the codebase, ensuring connections are always closed after use.

The initializeDatabase() method is called once at application startup from the main method. It executes five CREATE TABLE IF NOT EXISTS statements, one for each table in the schema. The IF NOT EXISTS clause means this method is safe to call every time the application starts — it will create the tables on the first run and do nothing on subsequent runs. This eliminates the need for any manual database setup by the user.

11.2 UserAuth Module

The UserAuth class implements all authentication logic. Its most important private method is hashPassword, which takes a plaintext string and returns its SHA-256 hash as a hexadecimal string.

The SHA-256 implementation uses Java's java.security.MessageDigest class. The process is: obtain a MessageDigest instance for the "SHA-256" algorithm, call digest() with the password bytes to get a byte array of the hash, then convert each byte to a two-character hexadecimal string and concatenate them. The result is a 64-character hexadecimal string that uniquely represents the input but cannot be reversed to recover the original input.

The registerUser method takes all five registration fields, hashes the password and security answer, and inserts a new row into the users table using a PreparedStatement. If the insert fails — most likely because the username already exists, triggering the UNIQUE constraint — the SQLException is caught and the method returns false, which the GUI uses to display an appropriate error message.

The login method takes a username and password, hashes the password, and queries the users table for a row matching both. If a matching row is found, the method returns true and the GUI proceeds to show the dashboard. If not, it returns false and the GUI shows an error message.

The getSecurityQuestion method takes a username and returns the security_question field for that user, or null if the user is not found. This is used in the forgot-password flow to display the question to the user before asking for the answer.

The verifySecurityAnswer method takes a username and an answer, hashes the answer, and checks it against the stored hash. This is the second step of the forgot-password flow.

The resetPassword method takes a username and a new password, hashes the new password, and updates the password field in the users table. It returns true if the update affected at least one row.

The generateTempPassword method generates a random 8-character alphanumeric string using Java's Random class. This temporary password is displayed to the user after a successful password reset and is immediately hashed and stored in the database.

11.3 ProductManager Module

The ProductManager class handles all product-related database operations. It also defines the Product class as a data transfer object.

The addProduct method takes all product fields and inserts a new row into the products table using a PreparedStatement with seven parameter placeholders. Using PreparedStatement rather than string concatenation is important for both security (preventing SQL injection) and correctness (handling special characters in product names correctly).

The updateProduct method takes a product ID and all updatable fields, and executes an UPDATE statement. It also

sets the updated_at field to CURRENT_TIMESTAMP so the database always reflects when a product was last modified. The method returns true if the update affected at least one row, false otherwise.

The deleteProduct method takes a product ID and executes a DELETE statement. It returns true if the deletion was successful.

The getAllProducts method executes a SELECT * FROM products ORDER BY name query and iterates through the ResultSet, constructing a Product object for each row and adding it to a List. The list is returned to the caller. Ordering by name makes the product list easier to navigate in the GUI.

The getProductById method takes an ID and returns a single Product object, or null if no product with that ID exists.

The updateProductQuantity method is specifically designed for use during sale processing. It takes a product ID and a quantity change (which will be negative when stock is being decremented on sale) and executes an UPDATE that adds the change to the current quantity. Using a relative update (quantity = quantity + ?) rather than an absolute set (quantity = ?) is important for correctness in concurrent scenarios, though in this single-user desktop application it is primarily a matter of good practice.

The getLowStockProducts method executes a SELECT query with a WHERE clause of quantity <= min_stock_level, ordered by quantity ascending so the most critically low products appear first. This query is the heart of the low-stock alert feature.

11.4 SalesManager Module

The SalesManager class contains the most complex logic in the application. The createSale method is the centerpiece and deserves detailed explanation.

The method begins by obtaining a database connection and immediately setting autoCommit to false. This is the start of the database transaction. From this point, no changes will be committed to the database until conn.commit() is explicitly called. If anything goes wrong before that point, conn.rollback() will undo all changes.

The method then calculates the financial totals. It iterates through the list of SaleItem objects passed as a parameter, summing their total prices to get the subtotal. It calculates tax as 10% of the subtotal. It calculates the final amount as subtotal plus tax minus discount.

Next, it generates a bill number by calling generateBillNumber(), which formats the current date and time as "BILL-yyyyMMddHHmmss". This produces a bill number like "BILL-20241215143022" that is both unique (because it includes seconds) and human-readable.

The method then inserts the sale record into the sales table using a PreparedStatement with RETURN_GENERATED_KEYS to retrieve the auto-generated sale ID. This ID is needed for the sale_items inserts.

It then iterates through the sale items again, using a batch insert to add all line items to the sale_items table in a single database round-trip. For each item, it also calls ProductManager.updateProductQuantity with a negative quantity to decrement the stock.

After all items are processed, it inserts an income entry into the accounts table, recording the final sale amount with a description of "Sale - " followed by the bill number and a category of "sales". This is the automatic financial recording that keeps the accounts ledger in sync with sales.

Finally, it calls conn.commit() to commit the entire transaction. If any step threw a SQLException, the catch block calls conn.rollback() to undo everything, and the method returns null to indicate failure. The finally block ensures the connection is always closed and autoCommit is restored to true.

This transactional approach guarantees that the database is always in a consistent state. Either all five operations succeed together (sale record, all line items, all stock updates, income entry) or none of them do.

11.5 AccountManager Module

The AccountManager class handles financial transaction recording and querying. Its most interesting method is getTransactions, which supports dynamic filtering.

The method builds a SQL query string starting with SELECT * FROM accounts WHERE 1=1. The WHERE 1=1 trick is a common pattern that allows additional AND clauses to be

appended conditionally without worrying about whether the WHERE keyword has already been used. If a type filter is specified and is not "all", a clause filtering by transaction_type is appended. If a period filter is specified, a date range clause is appended using SQLite's DATE() function.

The getBalance method uses a single SQL query with a CASE expression inside a SUM to calculate income and expense totals in one pass and return their difference. The query is: SELECT SUM(CASE WHEN transaction_type = 'income' THEN amount ELSE 0 END) - SUM(CASE WHEN transaction_type = 'expense' THEN amount ELSE 0 END) as balance FROM accounts. This is an efficient approach that avoids two separate queries.

11.6 GUI Module

The GUI is built entirely with Java Swing and organized around a CardLayout. The main window is a JFrame sized at 1000x700 pixels, centered on screen. The CardLayout panel holds all screens as named cards.

Each screen is created by a dedicated private method. The createLoginPanel method builds the login screen with a header panel, a centered login form with username and password fields, and three buttons. The createRegisterPanel method builds the registration screen with a five-field form. The createForgotPasswordPanel method builds the password recovery screen with a two-step flow.

The createMainDashboardPanel method builds the dashboard with a header showing the current username and

a logout button, a 2x3 grid of navigation buttons, and a stats bar at the bottom. Each navigation button has a distinct background color from the color palette and triggers navigation to the corresponding screen.

The createProductManagementPanel method builds the product management screen with a toolbar containing Add, Refresh, and Low Stock Alert buttons, and a JTable for displaying products. The table is wrapped in a JScrollPane to handle overflow.

The createSalesPanel method builds the sales screen as a JTabbedPane with two tabs: New Sale and Sales History. The New Sale tab has a customer details panel, a product selection table, a bill summary panel, and action buttons. The Sales History tab has a date filter panel and a sales table.

The createAccountsPanel method builds the accounts screen as a JTabbedPane with three tabs: Add Transaction, View Transactions, and Financial Summary.

The createReportsPanel method builds the reports screen as a JTabbedPane with three tabs for sales, inventory, and financial reports.

Helper methods reduce code duplication throughout the GUI. The createStyledButton method creates a JButton with a specified background color, white text, no focus border, and a hand cursor. The styleButton method applies similar styling to an existing button. The addFormField method adds a label-component pair to a GridBagLayout panel. The createHeaderPanel method creates a consistent header

panel with a back button and title for all non-dashboard screens. The showMessage method wraps JOptionPane.showMessageDialog for consistent message display.

12. Key Design Decisions

12.1 Choosing SQLite Over Other Databases

One of the earliest and most consequential decisions in this project was the choice of database engine. The options considered were MySQL, PostgreSQL, H2, and SQLite. MySQL and PostgreSQL are powerful, production-grade relational databases, but they require a separate server process to be installed, configured, and running before the application can function. This is a significant barrier for the target user — a small business owner who is not technically sophisticated. Asking them to install and manage a database server is unrealistic.

H2 is an embedded Java database that runs inside the JVM without a separate server, which makes it more accessible. However, H2 stores its data in a proprietary format and requires the H2 JAR as a dependency. SQLite was ultimately chosen because it is the most widely deployed database engine in the world, its file format is stable and well-documented, its JDBC driver is a single JAR file that is easy to include, and its database files are standard binary files that can be opened and inspected with freely available tools like DB Browser for SQLite. The portability and simplicity of SQLite perfectly matched the requirements of this project.

The trade-off is that SQLite is not designed for high-concurrency multi-user access. It uses file-level locking, which means only one write operation can occur at a time. For a single-user desktop application, this is completely acceptable. If the system were ever extended to support multiple simultaneous users over a network, a client-server database like PostgreSQL would be more appropriate.

12.2 SHA-256 for Password Hashing

The decision to use SHA-256 for password hashing was made with full awareness that it is not the ideal choice for a production security system. Modern best practice for password hashing recommends using a slow, memory-hard algorithm such as bcrypt, scrypt, or Argon2. These algorithms are specifically designed to be computationally expensive, making brute-force attacks impractical even with modern hardware.

SHA-256, by contrast, is a fast general-purpose cryptographic hash function. It is not designed for password hashing and can be brute-forced relatively quickly with dedicated hardware. However, for the context of this project — a desktop application for a small business, running on a local machine, with a database file that is not exposed to the internet — SHA-256 provides a reasonable level of protection. It ensures that passwords are not stored in plaintext, which is the most important baseline requirement.

The decision was also influenced by the fact that SHA-256 is available in the Java standard library through java.security.MessageDigest, requiring no external dependencies. Implementing bcrypt would have required adding a third-party library like jBCrypt, which adds complexity to the build and deployment process.

In a future version of this application, or in any version intended for broader deployment, the password hashing should be upgraded to bcrypt or Argon2. The upgrade path is straightforward: replace the hashPassword method in

UserAuth with a bcrypt implementation, and add a migration step that re-hashes existing passwords on first login.

12.3 CardLayout for Multi-Screen Navigation

The decision to use CardLayout for managing multiple screens was made after considering the alternatives. The most common alternative is to use multiple JFrame windows — opening a new window for each screen and closing the old one. This approach is simple to implement but produces a poor user experience: multiple windows appearing and disappearing on the taskbar, inconsistent window sizes, and no smooth transitions.

Another alternative is to use JDialog for secondary screens, keeping the main window open and showing dialogs on top of it. This works well for simple forms but becomes awkward for complex screens like the product management panel or the sales screen.

CardLayout was chosen because it keeps everything in a single window, provides instant screen transitions, and allows all screens to be pre-built at startup so there is no delay when navigating. The trade-off is that all screens consume memory even when not visible, but for an application of this size this is completely negligible.

The implementation creates all panels at startup and adds them to the CardLayout with string identifiers. Navigation is then a single method call: cardLayout.show(mainPanel, "SCREEN_NAME"). This is clean, fast, and easy to understand.

12.4 Static Methods in Manager Classes

All methods in the manager classes (UserAuth, ProductManager, SalesManager, AccountManager) are static. This means they can be called directly on the class without creating an instance: UserAuth.login(username, password) rather than new UserAuth().login(username, password).

This decision was made for simplicity. The manager classes have no instance state — they do not hold any data between method calls. Every method opens a database connection, performs its operation, closes the connection, and returns a result. There is no reason to create instances of these classes, so making the methods static eliminates unnecessary object creation and makes the calling code cleaner.

The trade-off is that static methods are harder to mock in unit tests, which makes test-driven development more difficult. In a larger project with a dedicated testing strategy, the manager classes would be better designed as interfaces with implementations that can be injected and mocked. For this project, the simplicity of static methods was the right trade-off.

12.5 Capturing Product Name and Price in Sale Items

A subtle but important design decision was to store the product_name and unit_price directly in the sale_items table at the time of sale, rather than relying on a join to the products table to retrieve them later.

The reason for this is historical accuracy. If a product's name is changed after a sale, or if its price is updated, the historical bill should still show the original name and price at the time of sale. If sale_items only stored the product_id and relied on a join to products to get the name and price, then any subsequent change to the product record would retroactively change the historical bill — which is incorrect and potentially fraudulent.

By capturing the name and price at sale time, the sale_items table becomes a permanent, immutable record of what was sold, at what price, at what time. This is the correct approach for any billing or accounting system.

12.6 Automatic Income Recording on Sale

The decision to automatically insert an income entry into the accounts table every time a sale is completed was made to keep the financial ledger in sync with sales without requiring any manual action from the user. When a sale is completed, the system automatically records the final sale amount as an income transaction with a description linking it to the bill number.

This means the financial summary in the Account Management module always reflects all sales, not just manually entered transactions. The user does not need to remember to record each sale as income — it happens automatically as part of the sale creation transaction.

This automatic recording is done within the same database transaction as the sale itself, so it is guaranteed to either succeed together with the sale or be rolled back together

with it. There is no possibility of a sale being recorded without a corresponding income entry, or an income entry being recorded without a corresponding sale.

12.7 Per-Product Minimum Stock Level

Rather than implementing a single global low-stock threshold that applies to all products, the decision was made to give each product its own min_stock_level field. This allows much more nuanced and useful low-stock alerting.

Different products have very different reorder dynamics. A fast-moving product like bottled water in a grocery store might need to be reordered when stock drops below 50 units. A slow-moving specialty item might only need reordering when it drops to 2 units. A single global threshold cannot accommodate this diversity.

By storing min_stock_level per product, the business owner can set appropriate thresholds for each item based on their knowledge of how quickly it sells and how long it takes to reorder. The low-stock query simply compares each product's current quantity against its own threshold, making the alert genuinely useful rather than generating false positives for slow-moving items or missing alerts for fast-moving ones.

12.8 Transaction Management in Sale Creation

The decision to wrap the entire sale creation process in a single database transaction was one of the most important technical decisions in the project. The sale creation process touches three tables: it inserts into sales, inserts multiple

rows into sale_items, and updates multiple rows in products. It also inserts into accounts. That is potentially many individual database operations that all need to succeed or fail together.

Without a transaction, a failure partway through — for example, a disk error after the sale record is inserted but before the stock quantities are updated — would leave the database in an inconsistent state. The sale would be recorded but the stock would not be decremented, meaning the inventory count would be wrong. Or the stock would be decremented but the income would not be recorded, meaning the financial summary would be wrong.

With a transaction, none of these partial-failure scenarios are possible. Either everything succeeds and is committed, or everything fails and is rolled back. The database is always in a consistent state.

13. Challenges Faced and How They Were Resolved

13.1 Challenge: Keeping the GUI in Sync with the Database

One of the most persistent challenges during development was ensuring that the data displayed in the GUI always reflected the current state of the database. This is a fundamental challenge in any application that combines a GUI with a database backend.

The specific manifestation of this problem was in the product management panel. When a product was added, edited, or deleted, the JTable displaying the product list needed to be refreshed to show the updated data. Similarly, after a sale was completed, the product quantities displayed in the product selection table needed to be updated.

The initial approach was to populate the tables once when the panel was created. This worked for the initial display but meant the data became stale as soon as any changes were made. The solution was to implement a refresh mechanism: a loadProducts() method that queries the database and rebuilds the table model, called both when the panel is first shown and whenever a modification is made. The Refresh button in the product management toolbar explicitly calls this method, giving the user manual control over when to reload data.

A more sophisticated solution would use a database change listener or an observer pattern to automatically refresh the

GUI whenever the underlying data changes. This was considered but deemed too complex for the scope of this project. The manual refresh approach is simple, reliable, and familiar to users of desktop applications.

13.2 Challenge: Managing Database Connections Correctly

SQLite with JDBC requires careful connection management. Each connection to the database opens a file handle to the inventory.db file. If connections are not properly closed after use, the file handles accumulate, eventually causing the application to run out of file descriptors or causing the database file to become locked.

The solution was to use Java's try-with-resources statement consistently throughout all manager classes. The try-with-resources statement guarantees that the Connection, PreparedStatement, and ResultSet objects are closed when the try block exits, whether normally or due to an exception. This pattern is used in every method that accesses the database.

The one exception is the createSale method in SalesManager, which cannot use try-with-resources for the connection because it needs to control the transaction manually — calling setAutoCommit(false), then either commit() or rollback(), then setAutoCommit(true) before closing. This method uses a traditional try-catch-finally block instead, with the finally block ensuring the connection is always closed and autoCommit is always restored.

13.3 Challenge: Component Reference in Anonymous Action Listeners

A significant challenge arose in the registration panel. The form fields (username, password, email, security question, security answer) were added to the form panel using the addFormField helper method, which takes a label string and a component and adds them to the panel. The problem was that the JButton's ActionListener needed to read the values from these fields, but the fields were local variables inside the addFormField calls and were not accessible from within the anonymous ActionListener.

In the Main.java version of the GUI, this was solved by declaring the field variables before calling addFormField and passing the pre-created components to the method. This kept the references accessible in the enclosing scope where the ActionListener was defined.

In the InventoryManagementGUI.java version, a different (and less clean) approach was used: the ActionListener accessed the form components by index from the parent panel's component array using registerForm.getComponents(). This works because the components are added to the panel in a known order, so component at index 1 is the username field, index 3 is the password field, and so on. While this approach works, it is fragile — if the order of components ever changes, the indices become wrong and the wrong values are read. This was noted as a technical debt item to be refactored.

The lesson learned was that when building forms in Swing, it is always better to keep explicit references to input

components rather than trying to retrieve them later by traversal.

13.4 Challenge: SQL Injection Vulnerability in AccountManager

During a code review of the AccountManager class, a SQL injection vulnerability was identified in the getTransactions method. The method builds a SQL query string by concatenating the type filter value directly into the query string:

```
sql += " AND transaction_type = '" + type + "'";
```

If the type value came from user input that was not properly controlled, an attacker could inject arbitrary SQL by providing a value like: income' OR '1'='1

In this specific case, the type value comes from a JComboBox in the GUI that only allows the values "All", "Income", and "Expense", so the actual injection risk is low. However, the pattern is still wrong and should not be used.

The correct solution is to use a PreparedStatement with a parameter placeholder for the type value, just as is done in all other methods. The challenge with this particular method is that the query is built dynamically — the WHERE clause may or may not include the type filter depending on the input. PreparedStatement does not support optional parameters natively.

The resolution for this version was to document the issue clearly and note it as a known limitation. The proper fix for a future version is to build the query with a fixed set of parameter placeholders and use a different PreparedStatement for each combination of filters, or to use a query builder library that handles dynamic parameterized queries cleanly.

13.5 Challenge: Duplicate GUI Entry Points

During development, two nearly identical GUI classes were created: Main.java and InventoryManagementGUI.java. This happened because the project went through an iterative development process where a second version of the GUI was started to incorporate improvements, but the original was never deleted. Both files contain essentially the same code with minor differences.

This duplication is a form of technical debt. It means that any bug fix or improvement to the GUI needs to be applied in two places, doubling the maintenance burden. It also creates confusion about which file is the actual entry point.

The resolution for this version was to designate Main.java as the authoritative entry point and note InventoryManagementGUI.java as a legacy file. In a future refactoring, InventoryManagementGUI.java would be deleted and any improvements from it would be merged into Main.java.

13.6 Challenge: Placeholder Data in Dashboard Stats

The dashboard stats bar at the bottom of the dashboard panel displays four metrics: total products, low stock count, today's sales, and monthly revenue. In the current implementation, these values are hardcoded as placeholder strings ("Total Products: 45", "Low Stock: 3", etc.) rather than being fetched from the database.

This happened because the stats panel was built early in development as a visual placeholder, with the intention of wiring it up to live database queries later. The live query implementation was not completed within the project timeline.

The fix is straightforward: replace the hardcoded strings with calls to ProductManager.getAllProducts().size() for total products, ProductManager.getLowStockProducts().size() for low stock count, and SQL aggregate queries on the sales and accounts tables for today's sales and monthly revenue. This would be one of the first things addressed in the next development iteration.

13.7 Challenge: Handling the Sale Transaction Across Multiple Connections

An interesting technical challenge arose in the createSale method. The method needs to update product quantities as part of the sale transaction, but the updateProductQuantity method in ProductManager opens its own database connection internally. This means the stock update happens on a different connection than the sale transaction, which means it is not part of the same transaction and cannot be rolled back if the sale fails.

This is a subtle but real problem. If the sale record and line items are inserted successfully but then the stock update fails, the rollback will undo the sale record and line items but the stock update (which happened on a different connection) will not be rolled back.

The resolution in the current implementation is that updateProductQuantity is called within the try block of the transaction, and if it fails, the catch block rolls back the sale-related changes. However, the stock update itself is not rolled back because it was on a different connection. This is a known limitation.

The proper fix is to pass the existing connection into the updateProductQuantity method so that the stock update participates in the same transaction. This would require changing the method signature to accept a Connection parameter, which is a straightforward refactoring but was not completed in this version.

13.8 Challenge: Bill Number Uniqueness

The bill number generation uses a timestamp formatted as "BILL-yyyyMMddHHmmss". This produces a unique bill number as long as no two sales are completed within the same second. For a single-user desktop application processing sales one at a time, this is entirely sufficient.

However, if the application were ever extended to support multiple simultaneous users or high-volume sales processing, two sales could theoretically be completed in the same second, producing duplicate bill numbers. The UNIQUE constraint on the bill_number column would catch

this and throw a SQLException, but the user experience would be poor.

A more robust solution would use a UUID (Universally Unique Identifier) for bill numbers, or a database sequence that guarantees uniqueness regardless of timing. For the current single-user context, the timestamp approach is adequate.

14. Testing and Validation

14.1 Testing Approach

The testing approach for this project was primarily manual functional testing, supplemented by code review and logic tracing. Automated unit tests were not written as part of this project, which is acknowledged as a limitation. In a professional development context, unit tests would be written for all manager class methods, and integration tests would verify the end-to-end flows.

14.2 Authentication Testing

The authentication module was tested by attempting to register with valid data and verifying that the user record appeared in the database with a hashed password. Registration was then attempted with the same username to verify that the duplicate rejection worked correctly. Login was tested with correct credentials, incorrect password, and non-existent username. The forgot-password flow was tested end-to-end: registering a user, using the forgot-password screen to retrieve the security question, providing the correct answer, receiving a temporary password, and logging in with that temporary password.

14.3 Product Management Testing

Product management was tested by adding products with various data types and verifying they appeared correctly in the table. Products were edited and the changes verified in

the database. Products were deleted and confirmed to be removed. The low-stock alert was tested by setting a product's minimum stock level higher than its current quantity and verifying it appeared in the low-stock list.

14.4 Sales Processing Testing

The sales module was the most thoroughly tested because it is the most complex. A sale was created with multiple products and the following were verified: the sale record appeared in the sales table with correct totals, the line items appeared in the sale_items table with correct quantities and prices, the product quantities in the products table were decremented by the correct amounts, and an income entry appeared in the accounts table with the correct amount and bill number reference.

The transaction rollback was tested by temporarily introducing an error condition (a product ID that does not exist) and verifying that none of the partial changes were committed to the database.

14.5 Financial Module Testing

The accounts module was tested by adding income and expense transactions and verifying that the financial summary calculations were correct. The filter functionality was tested with different type and period combinations. The balance calculation was verified by manually computing the expected balance from the transaction records and comparing it to the displayed value.

14.6 GUI Testing

The GUI was tested by navigating through all screens and verifying that all buttons, forms, and navigation elements worked correctly. Error messages were verified to appear when required fields were left empty. Success messages were verified to appear after successful operations. The back navigation from all screens was tested to ensure it returned to the correct previous screen.

---

15. What I Learned

15.1 The Value of Database-First Design

One of the most important lessons from this project was the value of designing the database schema before writing any application code. In earlier programming projects, the tendency was to start writing code immediately and figure out the data model as needed. This project demonstrated why that approach leads to problems.

When the schema is designed first, every subsequent decision — what methods to write, what data to pass between layers, what the GUI needs to display — is grounded in a clear understanding of the data model. Changes to the schema late in development are expensive because they ripple through all the code that depends on it. Getting the schema right at the start saves enormous time and effort.

The process of designing the schema also forced a deeper engagement with the problem domain. Thinking carefully about what data needs to be stored, how entities relate to each other, and what queries will need to be run against the data is essentially the same as thinking carefully about the business requirements. The schema design process was simultaneously a requirements analysis process.

15.2 JDBC and Relational Database Programming

This project provided extensive hands-on experience with JDBC that went far beyond what is covered in textbook examples. Working with JDBC in a real application context

meant dealing with connection lifecycle management, PreparedStatement parameterization, ResultSet iteration, transaction control, and error handling — all in the context of a system where correctness actually matters.

The most valuable JDBC lesson was understanding the difference between Statement and PreparedStatement. Statement is simple but vulnerable to SQL injection and handles special characters poorly. PreparedStatement is slightly more verbose but is the correct choice for any query that includes user-supplied data. Using PreparedStatement consistently throughout the project was both a security practice and a correctness practice.

The transaction management in the createSale method was particularly educational. Understanding why transactions are necessary, how to use setAutoCommit(false) and commit() and rollback() correctly, and what happens when a transaction fails — these are concepts that are easy to understand in theory but much more deeply understood through implementation.

15.3 Java Swing Layout Management

Building a multi-screen desktop application from scratch provided deep practical experience with Java Swing layout managers. The three layout managers used most extensively in this project were GridBagLayout, BorderLayout, and CardLayout, each serving a different purpose.

GridBagLayout is the most powerful and most complex layout manager in Swing. It allows precise control over the placement and sizing of components using a grid of cells

with configurable spanning, padding, and alignment. Mastering GridBagConstraints — understanding what each field does and how they interact — took significant time and experimentation. The lesson learned is that GridBagLayout is the right choice for complex forms but requires patience and careful attention to the constraint settings.

BorderLayout is simple and powerful for dividing a panel into five regions: north, south, east, west, and center. It was used extensively for the overall structure of each screen — a header panel in the north, a content panel in the center, and action buttons in the south.

CardLayout was the key to the multi-screen navigation architecture. Understanding how to add panels with string identifiers and switch between them with a single method call was a clean and effective solution to the screen management problem.

15.4 Cryptographic Security in Practice

Implementing SHA-256 password hashing gave practical experience with cryptographic security concepts that are often taught abstractly. The key insight is that a cryptographic hash function is a one-way transformation: given a password, you can always compute its hash, but given a hash, you cannot recover the original password. This property is what makes hashing suitable for password storage.

The implementation using Java's MessageDigest class was straightforward, but understanding why each step is necessary — why the byte array needs to be converted to a

hexadecimal string, why the hash is 64 characters long, why the same input always produces the same output — required engaging with the underlying mathematics at a conceptual level.

The research done around this topic also introduced the concept of salting — adding a random value to the password before hashing to prevent rainbow table attacks. While salting was not implemented in this version, understanding why it is important and how it works was a valuable learning outcome.

15.5 Software Architecture and Separation of Concerns

Perhaps the most broadly applicable lesson from this project was the practical value of separation of concerns in software architecture. The decision to separate the GUI, business logic, and data access into distinct layers — and to enforce that separation by never putting SQL in the GUI and never putting layout code in the manager classes — made the codebase dramatically easier to work with.

When a bug was found in the sales calculation, only SalesManager.java needed to be examined. When a GUI layout issue needed to be fixed, only the GUI class needed to be touched. When the database schema needed to be changed, only DatabaseConnection.java and the affected manager class needed to be updated. This isolation of concerns meant that changes were local and predictable, rather than requiring changes scattered across multiple files.

This experience made the abstract principle of separation of concerns very concrete. It is not just a theoretical best practice — it has a direct, measurable impact on how easy or difficult it is to understand, debug, and modify code.

15.6 The Importance of Error Handling

Building a real application that interacts with a database made the importance of error handling very concrete. Database operations can fail for many reasons: the database file might be locked, a constraint might be violated, the disk might be full, the query might be malformed. Every database operation in this project is wrapped in a try-catch block that handles SQLException.

The lesson learned was that error handling is not just about preventing crashes — it is about providing useful feedback to the user and maintaining the integrity of the application state. A bare exception that crashes the application is worse than useless. A caught exception that displays a clear error message and leaves the application in a consistent state is what users need.

The transaction rollback in createSale is the most sophisticated example of this. When a sale fails, the rollback ensures the database is left in exactly the state it was in before the sale was attempted. The user sees an error message and can try again. Nothing is corrupted or partially written.

15.7 Iterative Development and Technical Debt

The existence of two nearly identical GUI files (Main.java and InventoryManagementGUI.java) and the placeholder data in the dashboard stats panel are concrete examples of technical debt — shortcuts taken during development that work in the short term but create maintenance problems in the long term.

This project taught the lesson that technical debt is not always avoidable, especially under time pressure, but it needs to be consciously managed. The key is to be aware of where the debt exists, document it clearly, and have a plan for addressing it. Pretending the debt does not exist leads to it accumulating until it becomes a serious problem.

The practice of noting known limitations and future improvements — as done in this report and in the README — is a form of technical debt management. It creates a record of what needs to be done and why, which is valuable both for the original developer and for anyone who works on the code in the future.

15.8 Understanding the Problem Domain

A less technical but equally important lesson was the value of deeply understanding the problem domain before writing code. The design decisions that made this project work well — per-product minimum stock levels, automatic income recording on sale, capturing product name and price in sale items for historical accuracy — all came from thinking carefully about how a real business operates and what it actually needs.

A developer who does not understand the business domain will build a system that is technically correct but practically useless. Understanding why a business needs to track cost price separately from selling price, why historical bills must not change when product prices are updated, why a sale must be atomic — these insights come from domain knowledge, not from programming knowledge.

16. Future Improvements and Roadmap

16.1 Immediate Priority Improvements

The highest priority improvements are those that address known gaps in the current implementation.

The dashboard stats panel needs to be connected to live database queries. The four metrics — total products, low stock count, today's sales, and monthly revenue — should be fetched from the database every time the dashboard is shown, so they always reflect the current state of the business. This is a straightforward change that would significantly improve the usefulness of the dashboard.

The SQL injection vulnerability in AccountManager.getTransactions needs to be fixed by replacing the string-concatenated type filter with a parameterized PreparedStatement. This is a security fix that should be treated as high priority even though the current risk is low.

The duplicate GUI files need to be consolidated. InventoryManagementGUI.java should be deleted and any improvements from it merged into Main.java. This reduces maintenance burden and eliminates confusion about the entry point.

The connection sharing issue in createSale needs to be resolved by passing the transaction connection into updateProductQuantity so that stock updates participate in the same transaction as the sale record and line items.

16.2 Feature Additions — Short Term

In the short term, the following features would significantly improve the application's usefulness.

PDF bill generation would allow the business to print or email receipts to customers. This could be implemented using the iText or Apache PDFBox library to generate a formatted PDF from the bill data. The bill receipt text that is already generated by SalesManager.generateBillReceipt could serve as the content template.

CSV export for sales and financial reports would allow the business owner to open their data in Excel or Google Sheets for further analysis. Java's standard file I/O is sufficient for writing CSV files — no external library is needed.

Input validation throughout the GUI needs to be strengthened. Currently, some fields accept any text input without checking whether it is the correct type. Price and cost fields should only accept numeric input. Quantity fields should only accept positive integers. Required fields should be highlighted when left empty rather than just showing a generic error message.

The reports module needs to be connected to real data. The three report tabs currently show placeholder labels. They should be populated with actual data from the database — sales totals by date, product stock levels, income vs expense breakdowns.

16.3 Feature Additions — Medium Term

In the medium term, the following features would transform the application from a basic management tool into a comprehensive business platform.

Role-based access control would allow the business owner to create staff accounts with limited permissions. A staff member might be able to process sales and view products but not access financial reports or delete products. An admin account would have full access. This requires adding a role field to the users table and checking permissions before allowing access to each module.

Supplier management would add a module for recording supplier information and purchase orders. When new stock arrives, the owner could record a purchase order that automatically increments product quantities and records the cost as an expense in the accounts ledger. This would complete the inventory lifecycle from procurement to sale.

Customer management would add a customer database where repeat customers can be registered. Sales would be linked to customer records, enabling purchase history tracking, loyalty discounts, and customer-specific reporting.

Data visualization using JFreeChart would add bar charts, pie charts, and line graphs to the reports module. Visual representations of sales trends, product performance, and financial ratios are much more useful for business decision-making than raw numbers in a table.

16.4 Feature Additions — Long Term

In the long term, the following architectural improvements would prepare the application for broader deployment.

A network-capable version would allow multiple users on different computers in the same business to access the same database simultaneously. This would require migrating from SQLite to a client-server database like PostgreSQL or MySQL, and potentially moving to a client-server application architecture where the business logic runs on a server and the GUI clients connect over a local network.

A mobile companion application would allow the business owner to check stock levels, view sales summaries, and receive low-stock alerts on their smartphone. This could be built as an Android application that connects to the same database, or as a web application served from a local server.

Barcode scanning integration would allow products to be selected during a sale by scanning their barcodes rather than searching manually. This would require integrating with a barcode scanner device (most connect via USB and appear as keyboard input) and maintaining a barcode field in the products table.

Cloud backup would allow the inventory.db file to be automatically backed up to a cloud storage service like Google Drive or Dropbox, protecting against data loss from hardware failure. This could be implemented as a scheduled background task that copies the database file to a configured backup location.

17. Conclusion

This project set out to build a practical, integrated, offline-capable inventory management system for small businesses, and it has achieved that goal. The application delivers a complete set of core business management features — user authentication, product catalog management, sales processing with automatic billing, financial transaction tracking, and business reporting — in a single cohesive desktop application that requires no internet connection, no server setup, and no subscription fee.

The technical implementation demonstrates the application of a wide range of computer science concepts in an integrated, real-world context. The layered architecture cleanly separates the GUI, business logic, and data access concerns. The database schema accurately models the business domain with appropriate relationships and constraints. The transaction management in the sales module ensures data integrity even in failure scenarios. The SHA-256 password hashing protects user credentials from exposure. The CardLayout navigation provides a smooth, single-window user experience.

The challenges encountered during development — connection management, component reference in anonymous listeners, SQL injection risks, duplicate code, placeholder data — were all identified, analyzed, and either resolved or documented as known limitations with clear remediation paths. These challenges were not failures; they were learning opportunities that produced genuine insights about software engineering practice.

The lessons learned from this project go beyond the specific technologies used. The value of database-first design, the importance of separation of concerns, the practical implications of cryptographic security, the necessity of transaction management, the cost of technical debt — these are principles that apply to software development in any language, on any platform, for any domain.

The project also demonstrated the importance of understanding the problem domain deeply before writing code. The design decisions that make this system genuinely useful — per-product stock thresholds, automatic income recording, historical price capture in sale items — all came from thinking carefully about how a real business operates. Technical skill is necessary but not sufficient for building useful software; domain understanding is equally important.

Looking forward, the roadmap of improvements identified in Section 16 provides a clear path for evolving this application from a solid first version into a comprehensive business management platform. The foundation built in this project — the database schema, the layered architecture, the core business logic — is sound and extensible. Each improvement can be added incrementally without requiring a rewrite of the existing code.

In summary, this project successfully demonstrates the ability to conceive, design, implement, and evaluate a non-trivial software system that addresses a real-world problem. It is not a toy application or a tutorial exercise — it is a working system that could genuinely be used by a real business to manage their operations more effectively. That practical utility is the most meaningful measure of its success.

18. References

1. Bloch, J. (2018). Effective Java, Third Edition. Addison-Wesley Professional. — Referenced for Java best practices including the use of PreparedStatement, try-with-resources, and static utility methods.

2. Date, C. J. (2003). An Introduction to Database Systems, Eighth Edition. Addison-Wesley. — Referenced for relational database design principles, normalization, and transaction theory.

3. Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. — Referenced for the understanding of layered architecture and separation of concerns.

4. SQLite Documentation. (2024). SQLite Home Page. https://www.sqlite.org/docs.html — Referenced for SQLite data types, SQL syntax, transaction semantics, and PRAGMA settings.

5. Oracle Corporation. (2024). Java SE 8 API Documentation. https://docs.oracle.com/javase/8/docs/api/ — Referenced for Java Swing component documentation, JDBC API, MessageDigest, and Random class usage.

6. Xerial. (2024). SQLite JDBC Driver. https://github.com/xerial/sqlite-jdbc — The JDBC driver used to connect the Java application to the SQLite database.

7. OWASP Foundation. (2024). Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html — Referenced for understanding password hashing best practices and the limitations of SHA-256 for password storage.

8. OWASP Foundation. (2024). SQL Injection Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html — Referenced for understanding SQL injection risks and the use of parameterized queries as the primary defense.

9. Ministry of Micro, Small and Medium Enterprises, Government of India. (2023). Annual Report 2022-23. https://msme.gov.in — Referenced for statistics on the number and economic significance of MSMEs in India.

10. Horstmann, C. S. (2019). Core Java Volume I — Fundamentals, Eleventh Edition. Prentice Hall. — Referenced for Java Swing layout managers, event handling, and GUI component usage.

11. Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley Professional. — Referenced for the understanding of the three-layer architecture pattern

(presentation, domain logic, data source) applied in this project.

12. Koskela, L. (2013). Practical TDD and Acceptance TDD for Java Developers. Manning Publications. — Referenced for understanding the value of automated testing and the challenges of testing static methods, informing the acknowledgment of testing limitations in this project.